

Balanced Drag — Algorithm Pseudocode

Momentum-observer estimate of the hand torque + Cartesian inertia shaping toward an isotropic virtual mass at the gripper, with load-tracking friction compensation and sustained relief on joint 1. Companion to the “B601 Balanced Drag” derivation artifact; this document is the implementation, line for line, with the math behind each step.

Model. Six revolute joints; the plant and the hand wrench:

$$M(q)\ddot{q} + C(q, \dot{q})\dot{q} + g(q) + \tau_f(\dot{q}) = \tau_m + \tau_h, \quad \tau_h = J^T(q)F_h$$

$\tau_m = \tau_{ff} + k_p \cdot (\text{pos} - q) + k_d \cdot (\text{vel_cmd} - \dot{q})$ is the MIT frame we command — the RobStride torque echo is this command reflected back and carries no load information, so τ_h must be inferred from motion. The target the hand should feel is an isotropic virtual body at the gripper (tool frame; local x = roll = j6 axis):

$$\Lambda_d \ddot{x} = F_h, \quad \Lambda_d = \text{diag}(m_d, m_d, m_d, i_{roll}, i_{rot}, i_{rot}), \quad M_d(q) = J^T \Lambda_d J$$

Symbols and defaults

symbol / knob	meaning	default (CLI)
κ	max lightening ratio – 1; K eigenvalues clipped to $[-\rho, \kappa]$. $\kappa=0 \Rightarrow$ shaping off (toggles are independent; --observe = all off)	--balance κ , hard cap 2.0
ρ	resist floor: light directions made at most $1/(1-\rho) \times$ heavier	$\kappa/(1+\kappa)$ (--balance-resist)
m_d, i_{rot}, i_{roll}	target mass / rotational inertia at the gripper; live keys m / i / + / –	1.8 kg, 0.06, 0.0005 kg·m ²
$K_o = 2\pi f_o$	observer bandwidth	$f_o = 3$ Hz (--balance-fo)
$s_f; f_0, \mu; v_0, v_{st}$	friction scale; per-joint load model $f_0 + \mu \cdot g(q) $ for kinetic and static levels (config, from the f/s sweeps + fit_friction.py); tanh knee; Stribeck velocity	0.85 (--balance-fric); fit 2026-08-28; 0.08, 0.15 rad/s
sustain, m, v_h	joints whose relief persists while clearly moving; friction kept in reserve; motion threshold	joint1 (--balance-sustain), 0.20 N·m, 0.10 rad/s
r_0	dead-band on r ($\approx 2 \times$ gravity-fit residual rms)	[.10 .10 .10 .05 .05 .05] N·m
$\tau_{cap}, T_{ramp}, \lambda, \tau_b, r_{max}$	per-joint output clamp; ramp; damped-inverse λ ; bias time constant; $ r_{net} $ clamp	$0.4 \cdot \tau_{max}$, 2 s, 0.05, 5 s, 3 N·m
$g_{cal}, \hat{M}, C$	calibrated gravity ($g_{scale} \cdot g + g_{bias}$, absorbs k_t), URDF mass matrix (links only — no rotor inertia yet), Coriolis with $\dot{M} = C + C^T$	Pinocchio, per cycle

The math behind each step

1 • Observer (Algorithm 1). External torque from motion — no acceleration, no torque sensor.

$$p = M\dot{q}, \quad \dot{p} = \tau_m + \tau_h - \beta, \quad \beta = g_{cal}(q) - C^\top \dot{q} \quad (\text{friction deliberately not modelled})$$

$$\hat{p} \leftarrow \hat{p} + \Delta t (\tau_m - \beta + r), \quad r = K_o(p - \hat{p}) \implies r = \frac{K_o}{s + K_o} (\tau_h - \tau_f \text{sign}(\dot{q}) + \text{model error})$$

r is the low-passed *net drive beyond friction*, per joint. At rest $r \rightarrow 0$ (stiction hides everything); the slow bias learner absorbs what is left when the arm is still. The observer knows τ_m because we commanded it — the loop closes through the integral of past commands, so there is no algebraic loop and no $\ddot{q}$.

2 • Inertia shaping (Algorithm 3). Make every direction respond like the virtual body.

$$\tau_{out} = K(q)r_{net}, \quad K = \hat{M}M_d^{-1} - I \implies M_d \ddot{q} = \tau_h - \tau_f + \hat{f} \quad (\text{the hand feels } \Lambda_d)$$

$$\text{eig}(\hat{M}M_d^{-1}) = \text{eig}(\hat{\Lambda}\Lambda_d^{-1}), \quad 1\text{-DoF: } \tau = \left(\frac{m}{m_d} - 1\right) \hat{\tau}_h$$

Rows of $K \cdot r$ with the sign of the hand's torque on that joint **assist** — the real inertia there exceeds the target, and the motor accelerates the surplus (you push the virtual 1.8 kg, the motor pushes the rest in lockstep). Opposite-sign rows **resist** — the direction is naturally lighter than the target (the folding wrist) and the motor absorbs the excess response. The eigenvalue identity makes the joint-space gains a pure Cartesian statement: how much heavier than the target, direction by direction.

3 • Stability — why κ is capped at 2.

$$\dot{q} = \frac{1 + \kappa L(s)}{ms + b} \tau_h, \quad L = \frac{K_o}{s + K_o} \quad (\text{exact model: pure feed-forward — stable for any } \kappa)$$

$$\kappa < \frac{m}{\hat{m} - m} \quad (\hat{m} > m); \quad \kappa f_o \lesssim 8 \text{ Hz for 2 unmodelled cycles at 90 Hz} \implies \text{eig}(K) \in [-\rho, 2], \quad f_o = 3 \text{ Hz}$$

Every instability is proportional to model error: an inertia *over*-estimate feeds acceleration back positively (hence the bound), unmodelled delay binds at the loop rate, and an under-estimate (the URDF's missing rotor inertia) is the safe direction. The eigen-clip enforces the band; ramp, singularity fade (cond J 25→40) and the runaway guard cover the rest.

4 • Friction compensation (in Algorithm 2). Load-tracking level, Stribeck blend, drive gate.

$$f_{k,j}(q) = f_{0_{k,j}} + \mu_{k,j} |g_{cal,j}(q)|, \quad f_{s,j}(q) = f_{0_{s,j}} + \mu_{s,j} |g_{cal,j}(q)| \quad (\text{gear-mesh loss grows with load})$$

$$\ell_j = f_{k,j} + (f_{s,j} - f_{k,j}) e^{-(\dot{q}_j / v_{st})^2}, \quad gate_j = \text{clip}\left(\frac{r_j \text{sign}(\dot{q}_j)}{r_{0,j}}, 0, 1\right), \quad f_{ff,j} = s_f \ell_j \tanh(\dot{q}_j / v_0) gate_j$$

The gate exists to make compensation creep-proof: a self-driven joint has no hand behind it, so its $r = -f \cdot \text{sign}(\dot{q}) < 0$ and it shuts its own relief off. The price is a built-in contradiction: since $r = \tau_{hand} - f$, the gate opens only while the hand pushes *more* than full friction — the moment relief succeeds and the hand relaxes below f , r turns negative and the relief stops. Steady sliding therefore gets zero relief; the gate helps only in the over-pushing (accelerating) regime. Everywhere but j1 this is masked by inertia assist, long levers, gravity comp, or small absolute friction.

5 · Sustained relief on joint 1 (in Algorithm 2). The one joint where the gate's limit hurts and the fix is unconditionally safe.

$$F_{lateral} = \frac{\tau_{f,1}}{r} \quad \left(0.56 \text{ N}\cdot\text{m}: 1.4 \text{ N at } r = 0.4 \text{ m}, \quad 4.7 \text{ N at } r = 0.12 \text{ m} \right)$$

Lateral drags are steady, single-joint, assist-less (the lateral direction is lighter than the target) and gravity-free — friction is 100 % of the effort, amplified by the shrinking lever near the base. Fix: keep j1's relief on while it is clearly moving, capped a margin below its real friction:

$$\begin{aligned} sus_1 &= \text{clip} \left(\frac{|\dot{q}_1| - v_h}{v_h}, 0, 1 \right), & f_{sus} &= \min \left(s_f \ell_1, f_{k,1}(q) - m \right) \\ f_{ff,1} &= \tanh(\dot{q}_1 / v_0) \max \left(gate_1 \cdot s_f \ell_1, sus_1 \cdot f_{sus} \right) \end{aligned}$$

With the calibration: 0.476 N·m relief while demonstrably pushing, 0.36 N·m while merely sliding — steady lateral effort drops from 0.56 to 0.20 N·m at the joint (4.7 → ≈1.7 N close in). Safety invariants:

- (i) $|\dot{q}_1| < v_h \Rightarrow$ drive path only, and self-drive has $r_1 < 0$ (no bootstrap)
- (ii) $\underbrace{b + f_{sus} - f_{k,1}}_{\text{coasting net torque}} \leq b - m < 0 \quad \forall |b| \leq 0.15 < m = 0.20$

(iii) j1's axis is vertical: $g_1(q) \equiv 0$, so no gravity-model residual exists there, its friction is pose-constant (the load fit found nothing to model), and a coasting j1 holds no potential energy — the worst failure is a slow yaw that the margin plus kd damping stops in under a second. No other joint satisfies (iii), which is why sustain defaults to j1 only.

Controller integration (per ~90 Hz cycle of GravityDragController)

Algorithm 0 — hooks in the drag loop ($q, \dot{q}$ from MIT feedback, one cycle old)

1. **if** phase = DRAG: $\tau_{cmd} \leftarrow g_cal + \text{UPDATE}(q, \dot{q}, dt)$ // Alg. 2; config *fric_comp* is auto-disabled under *--balance*
2. **else**: **OBSERVE**($q, \dot{q}, dt$) // estimator stays live in RAMP/FADE/HOLD/CAPTURE; output forced 0
3. $\tau_{cmd} \leftarrow \text{clip}(\tau_{cmd}, \pm\tau_{max})$; send MIT frame ($pos, vel_{cmd}, kp, kd, \tau_{cmd}$)
4. **NOTE_SENT**($\tau_{cmd}, kp, kd, pos, vel_{cmd}$) // what was actually commanded, post-clip; used by Alg. 1 next cycle
5. log $r_1..r_6$ to the CSV; status line shows r and α // existing guards (*vel_abort→HOLD* etc.) unchanged

Algorithm 1 — OBSERVE: generalized-momentum estimate of the hand torque (in: $q, \dot{q}, dt \rightarrow r \in \mathbb{R}^6$)

```

1.  $M \leftarrow \hat{M}(q)$ ;  $C \leftarrow C(q, \dot{q})$ ;  $g\_cal \leftarrow \text{calibrated gravity}(q)$ ;  $p \leftarrow M \cdot \dot{q}$ 
2. if first call or nothing noted yet:  $\hat{p} \leftarrow p$  // anchor:  $r$  starts at zero
3. else:
4.    $(\tau\_ff, k_p, k_d, pos, vel\_cmd) \leftarrow \text{last NOTE\_SENT}$ ;  $\dot{q}\_mid \leftarrow \frac{1}{2}(\dot{q} + \dot{q}\_prev)$ 
5.    $\tau\_m \leftarrow \tau\_ff + k_p \cdot (pos - q) + k_d \cdot (vel\_cmd - \dot{q}\_mid)$  // torque the motors applied over the last interval
6.    $\hat{p} \leftarrow \hat{p} + dt \cdot (\tau\_m - (g\_cal - C^T \cdot \dot{q}) + r\_raw)$  // §1: integral form, no acceleration, no algebraic loop
7.    $r\_raw \leftarrow K_o \cdot (p - \hat{p})$  // low-pass of  $(\tau\_hand - \tau\_fric + \text{model error})$ 
8.   if all  $|\dot{q}| < 0.02$  rad/s for  $> 1$  s:  $bias \leftarrow bias + (r\_raw - bias) \cdot dt / \tau\_b$ ; clip  $\pm 0.5$  // at rest  $r$  is pure model bias
9.   return  $r \leftarrow r\_raw - bias$ 

```

Algorithm 2 — UPDATE: shaped output (in: $q, \dot{q}, dt \rightarrow \tau_out \in \mathbb{R}^6$; only called in DRAG)

```

1.  $r \leftarrow \text{OBSERVE}(q, \dot{q}, dt)$ 
2. if output was idle  $\geq 2$  cycles:  $t\_ramp \leftarrow 0$  // fresh DRAG stretch  $\Rightarrow$  ramp back in;  $t\_ramp \leftarrow t\_ramp + dt$ 
3.  $\ell \leftarrow \text{Stribeck level from } f_0 + \mu \cdot |g\_cal(q)| \text{ per joint}$  // §4: load-tracking, breakaway  $\rightarrow$  kinetic with speed
4.  $gate \leftarrow \text{clip}(r \cdot \text{sign}(\dot{q}) / r_0, 0, 1)$  per joint // §4: drive gate – creep-proof, but zero steady-state relief
5.  $sus \leftarrow \text{clip}((|\dot{q}| - v_h) / v_h, 0, 1)$  on sustain joints ( $j1$ ), else 0 // §5
6.  $f\_ff \leftarrow \tanh(\dot{q} / v_0) \cdot \max(gate \cdot s\_f \cdot \ell, sus \cdot \min(s\_f \cdot \ell, f\_k(q) - m))$  per joint // §5: margin-capped sustained relief
7.  $r\_db \leftarrow \text{sign}(r) \cdot \max(|r| - r_0, 0)$  // soft dead-band
8.  $r\_net \leftarrow \text{clip}(r\_db + f\_ff, \pm r\_max)$  // the drive the hand would have without the modelled friction
9.  $K, cond \leftarrow \text{SHAPING}(q)$ ;  $fade \leftarrow \text{clip}((40 - cond) / 15, 0, 1)$  // Alg. 3; singularity fade
10.  $\text{RUNAWAY}(\dot{q}, r\_net, dt)$  // Alg. 4; may halve  $s\_trip$  and restart the ramp
11.  $\alpha \leftarrow \min(1, t\_ramp / T\_ramp) \cdot fade \cdot s\_trip$ 
12. return  $\tau\_out \leftarrow \text{clip}(\alpha \cdot K \cdot r\_net + f\_ff, \pm \tau\_cap)$ 

```

Algorithm 3 — SHAPING: eigen-clipped gain matrix (in: $q \rightarrow K \in \mathbb{R}^{6 \times 6}, cond(J)$)

```

1.  $J \leftarrow \text{tool-frame Jacobian of gripper\_end } (6 \times 6)$ ;  $cond \leftarrow \sqrt{(\lambda\_max / \lambda\_min \text{ of } J \cdot J^T)}$ 
2.  $J^{-1} \leftarrow J^T \cdot (J \cdot J^T + \lambda^2 I)^{-1}$  // damped: finite at singularities
3.  $M\_d^{-1} \leftarrow J^{-1} \cdot \text{diag}(1 / \Lambda\_d) \cdot J^{-T}$  // desired joint-space mobility,  $M\_d = J^T \Lambda\_d J$ 
4.  $M^{\frac{1}{2}}, M^{-\frac{1}{2}}$  from  $\text{eigh}(\hat{M})$  //  $\hat{M}$  cached from Alg. 1 this cycle
5.  $S \leftarrow M^{\frac{1}{2}} \cdot M\_d^{-1} \cdot M^{\frac{1}{2}} - I$  // symmetric, similar to  $K = \hat{M} M\_d^{-1} - I \Rightarrow$  same (real) eigenvalues
6. eig-clip  $S$  to  $[-p, \kappa]$  // §2–3: the stability band, as a Cartesian ratio
7. return  $K \leftarrow M^{\frac{1}{2}} \cdot S\_clip \cdot M^{-\frac{1}{2}}$  //  $K > 0$  rows assist (arm heavier than target);  $K < 0$  rows resist (folding wrist)

```

Algorithm 4 — RUNAWAY: energy guard (in: $\dot{q}$, r_{net} , dt ; state: $s_{\text{trip}} \leftarrow 1$ at start)

1. $E \leftarrow \frac{1}{2} \cdot \dot{q}^T \cdot \hat{M} \cdot \dot{q}$
2. **if** E rising **and** $E > 0.02 \text{ J}$ **and** $\dot{q}^T \cdot r_{\text{net}} \leq 0.02 \text{ W}$: $t_{\text{run}} \leftarrow t_{\text{run}} + dt$ **else** $t_{\text{run}} \leftarrow 0$
// KE floor forgives the ~50 ms observer lag at push onset
3. **if** $t_{\text{run}} > 0.3 \text{ s}$: $\text{trips}++$; $s_{\text{trip}} \leftarrow s_{\text{trip}}/2$; $t_{\text{ramp}} \leftarrow 0$ *// energy appearing with no hand behind it $\Rightarrow$ shaping is driving itself*

WHY EACH GUARD EXISTS (from the derivation)

With an exact model the whole scheme is a feed-forward of the hand's own force — nothing to destabilise. Every risk is model error: an inertia *over*-estimate feeds acceleration back positively ($\kappa < m/(\hat{m}-m)$), and unmodelled delay binds at $\sim 90 \text{ Hz}$ ($\kappa \cdot f_o \approx 8 \text{ Hz}$ for two stray cycles) — hence $\kappa \leq 2$ with $f_o = 3 \text{ Hz}$, and the URDF's missing rotor inertia is tolerable because an *under*-estimate is the safe direction. Stiction stays invisible to any motion-based estimator: at rest $r \equiv 0$, so breakaway is unchanged and only sliding friction is compensated. Sustained relief exists only on j_1 , where invariant (iii) of §5 makes it unconditional; giving it to gravity-bearing joints reintroduces the self-motion the margin analysis forbids (observed once on j_5 , then reverted).

ROLLOUT / OPERATION Run: `drag --balance 1` (or 2). Defaults: 1.8 kg / 0.06 kg·m² target, 85 % load-tracking friction compensation from the config fit, sustained relief on joint1 (`--balance-sustain none` to disable). Calibrate friction from DRAG: f = kinetic sweep, s = per-joint breakaway ramp, at 4–6 spread poses (rows append to `friction.csv`), then `scripts/fit_friction.py` $\rightarrow f_o/\mu$ per joint, paste-ready config lines. Independent toggles: `--balance 0|1` (shaping), `--balance-fric 0.85|0`, `--balance-sustain 1|0`; `--observe` = all off, r logged (the safe first run). Live keys while dragging: m <kg> / i <kg·m²> set the virtual mass / rotational inertia, +/- step 25 % (slewed $\sim 0.5 \text{ s}$). Offline: `tests/test_balance.py`.